

7.2. Módulos (ESM)

En aplicaciones grandes, tener todo el código en un solo archivo JavaScript se vuelve confuso y complejo cuando el código crece.

En la mayoría de los lenguajes de programación, el **código se divide en archivos diferentes**, de modo que, por ejemplo, cada clase está en un archivo separado.

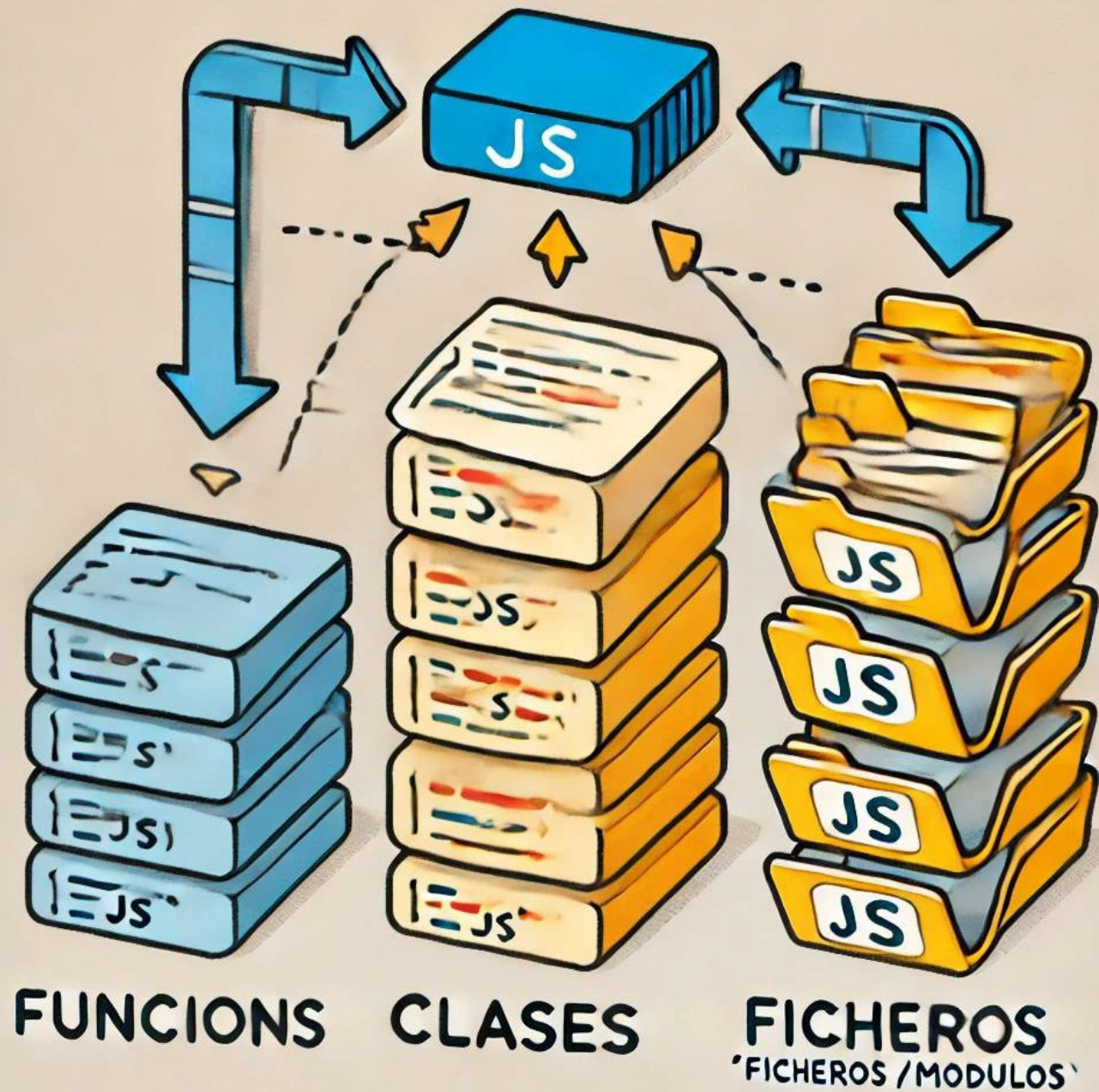


Esto hace que todo sea más **organizado, intuitivo y fácil de mantener**.

A partir de la versión ES2015, se introduce una característica nativa llamada **Módulos ES (ESM)**, que permite la **importación y exportación de fragmentos de datos entre diferentes archivos JavaScript**, permitiendo trabajar de forma más flexible.

Con **export** → creamos un módulo de exportación que contiene datos, como variables, funciones, clases u objetos complejos.

Con **import** → podemos leer módulos exportados y utilizar sus elementos en el archivo actual.



Por lo que disponemos de las siguientes palabras clave:

- **export**: pone datos (variables, funciones, clases...) a disposición de otros archivos.
- **import**: incorpora datos desde otros archivos **.js** al código actual.
- **import()**: permite importar módulos de forma dinámica, en tiempo real. (bajo demanda, si se necesita).

Para usar export o import, debemos cargar el archivo **.js** con la etiqueta **<script type="module">**.



index.html

html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Módulos JS</title>
</head>
<body>

  <script type="module" src="./app.js"></script>

</body>
</html>
```

👉 El navegador:

1. Carga `app.js` como módulo
2. Detecta el `import`
3. Carga automáticamente `math.js` como módulo
4. Ejecuta `math.js` una sola vez
5. Comparte sus exports



app.js

js

```
import { suma } from './math.js';

console.log(suma(2, 3));
```



math.js

js

```
export const suma = (a, b) => a + b;
```

→ los módulos se cargan por **dependencia**, no manualmente

Formas de exportar módulos

Existen varias formas de exportar datos con **export**:

- **export ...** : declara y añade un elemento al módulo de exportación.
- **export { name }** : añade el elemento name al módulo de exportación.
- **export { name as newName }** : añade el elemento name al módulo de exportación con el nombre newName.
- **export { n1, n2, n3... }** : añade los elementos n1, n2, n3, etc., al módulo de exportación.
- **export * from "./file.js"** : añade todos los elementos del módulo de file.js al módulo de exportación.

- **export * as name from "./file.js"**: añade todos los elementos del módulo de file.js en un objeto name.
- **export default ...** : declara un elemento y lo añade como el módulo de exportación por defecto.

Formas de importar módulos

Existen varias formas de importar datos con **import**:

- **import { name } from "./file.js"**: importa el elemento name de file.js.
- **import { name as newName } from "./file.js"**: importa el elemento name de file.js como newName.

- **import { n1, n2... } from "./file.js"**: importa los elementos indicados desde file.js.
- **import name from "./file.js"**: importa el elemento por defecto de file.js como name.
- **import * as name from "./file.js"**: importa todos los elementos de file.js en el objeto name.
- **import "./file.js"**: ejecuta el código de file.js. No importa ningún elemento.
- **import { name } from "https://web.com/file.js"**: descarga el fichero e importa el elemento name de su módulo.

Para poder trabajar con módulos se ha de utilizar un **servidor**, ya que cuando usamos **import** y **export**, el navegador bloquea la carga de archivos locales (**file://**) por seguridad.

```
file:///index.html ❌
```

7.2.1. Automatizadores

Anteriormente, crear un sitio web implicaba crear manualmente una carpeta para el proyecto, subcarpetas para secciones como imágenes, CSS y JS, configurar un servidor web y enlazar librerías CSS/JS.

Aunque aún es posible hacerlo, los **automatizadores** simplifican este proceso, especialmente mediante la **transpilación**.

Transpilar → transformar el código fuente de un lenguaje a otro código fuente diferente.

```
const add = (a, b) => a + b;  
console.log(add(2, 3));
```



```
let add = function(a, b) {  
  return a + b;  
};  
console.log(add(2, 3));
```

¿Por qué transpilar?

Para mejorar la compatibilidad y el rendimiento del código, aprovechando las características nuevas y en evolución del lenguaje.

Habitualmente, trabajamos en una carpeta **src/** donde tenemos nuestro código fuente.

Ese código se preprocesa y se traduce a otro código (en la carpeta **dist/**) que es el código que finalmente leerá nuestro navegador.

Mantener actualizado el código fuente que escribimos con el que lee el navegador sin automatizarlo, es una tarea muy tediosa. De ahí surgen los **automatizadores**.

Tareas de los automatizadores

- Creación y gestión del proyecto (generalmente, con NPM “gestor de paquetes para NodeJS”).
- Actualización y mantenimiento de dependencias (generalmente, con NPM).
- Servidor web local de desarrollo.
- Minificación de código.
- Optimización de imágenes.
- Preprocesamiento o transpilación del código fuente.
- Despliegue del código final al servidor.

Todas estas tareas pueden ser realizadas de forma manual, pero con el uso de un **automatizador**, lo hacemos automáticamente, de forma más rápida y con menos errores.

Automatizadores populares

Existen multitud de automatizadores: Browserify, Gulp, Webpack, Rollup, Grunt, Parcel, Snowpack, Esbuild, Vite, Turbopack, RSPack, Farm.

Algunos tienen una filosofía más de automatización de tareas (como Gulp) y otros tienen una filosofía más guiada como Vite, Webpack o Rollup.

Actualmente, los automatizadores más populares son Vite y Webpack, siendo **Vite** el más recomendado.

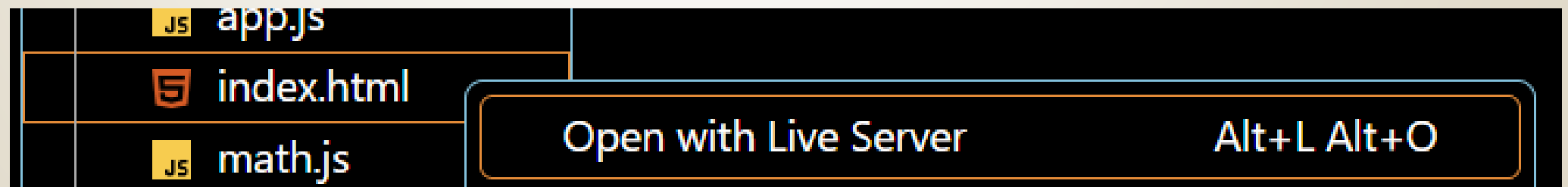
Y ahora que sabemos todo esto, ¿por dónde empezamos?

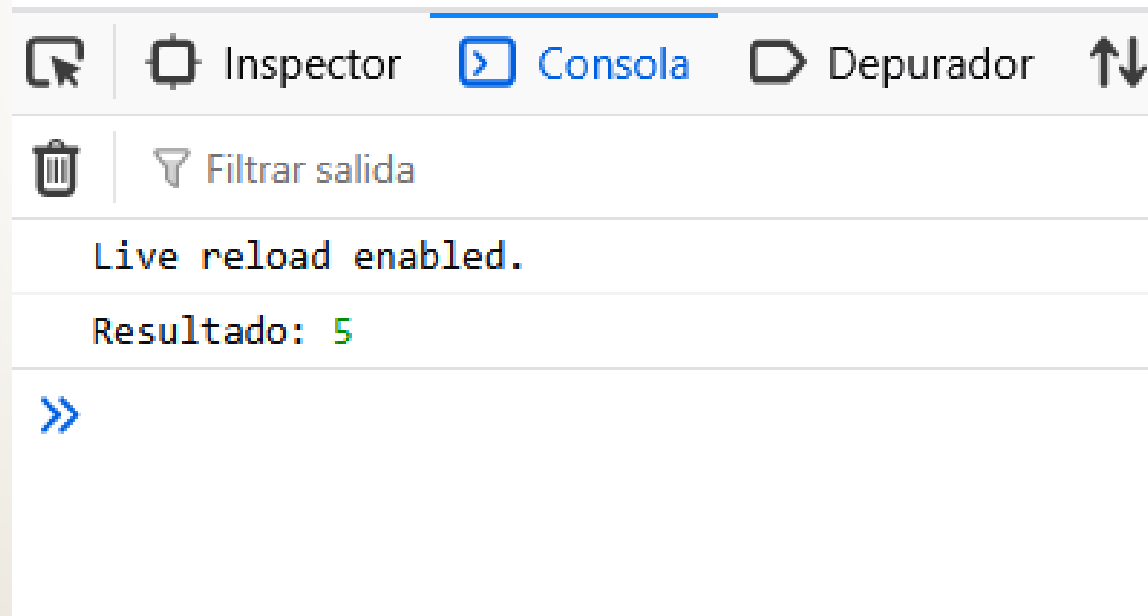
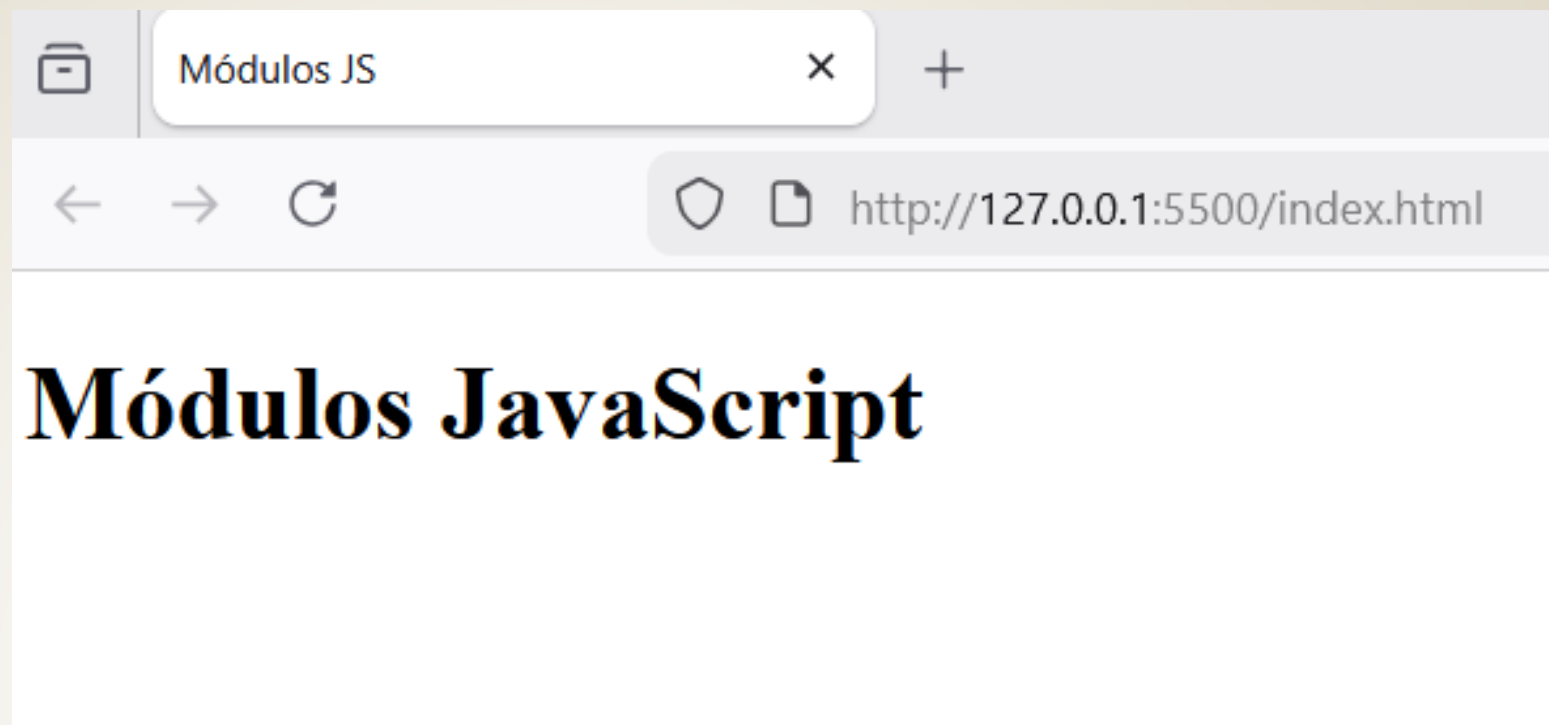
→ Si solo queremos **probar** cómo funciona “esto de los módulos” ...

En Visual Code tendremos:



... abriremos el index.html con **Live Server** que nos crea un servidor temporal para desarrollo local.





→ Si ya queremos gestionar un “**proyecto real**” ...

¡Instalaremos **NodeJS y NPM!**

NodeJS → se creó en 2009, como entorno multiplataforma, con la idea de expandir el uso de JavaScript más allá del navegador.

Hasta ese momento, JavaScript solo se podía utilizar dentro de navegadores web, pero NodeJS permitió **ejecutar JavaScript** en cualquier lugar fuera de estos, como **en servidores web**.

Esto transformó a JavaScript de ser solo un lenguaje de cliente a ser también un lenguaje de servidor, ampliando significativamente sus posibilidades y aplicaciones.

NPM, Node Package Manager → es un gestor de paquetes para NodeJS.

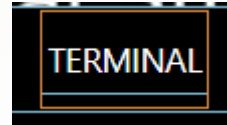
Facilita la instalación, gestión y mantenimiento de paquetes JavaScript, tanto para Node como para el navegador, en proyectos y sistemas.

Su funcionamiento es similar al de **apt-get** en GNU/Linux, proporcionando una **forma sencilla y automática de manejar dependencias en aplicaciones JavaScript**.

¿Tenemos NodeJS?

→ Podremos hacerlo desde el terminal del S.O. (Windows, Linux o Mac) o de Visual Code.

En Visual Code ...

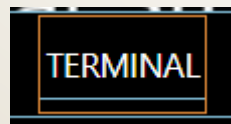


```
node -v
```

```
v22.15.1
```

Si no nos aparece la versión, descargamos NodeJS, lo instalamos en nuestro equipo y volvemos a abrir Visual Code para que lo detecte.

NodeJS normalmente viene con **npm**, que es el gestor de paquetes. Para verificarlo:



```
npm -v
```

```
11.4.0
```

lite-server y serve (en NPM)

lite-server es un servidor de desarrollo que sirve para:

- Programar
 - Guardar archivos
 - Ver cambios al instante
 - Trabajar cómodo
- Es una herramienta de trabajo, no de entrega.

serve es un servidor estático profesional que sirve para:

- Publicar una carpeta
 - Ver cómo quedará tu web en producción
 - Probar un build final (por ejemplo, de React o Vite)
- Es un “simulador de servidor real”.

lite-server y serve ...

- No son competencia.
- Se usan en momentos distintos del proyecto.

● Lite-server — Modo práctica / clase

- ♦ Recarga automática:  Sí, se actualiza al guardar
- ♦ Ideal para aprender módulos:  Sí
- ♦ Uso principal: Desarrollo (HTML + JS + CSS)
- ♦ URL por defecto:  `http://localhost:3000`
- ♦ Instalación: `npm install lite-server --save-dev`
- ♦ Script npm: `"start": "lite-server"`

💡 *Tip:* Perfecto para ver resultados al instante y evitar errores de caché.

● Serve — Modo real / producción

- ♦ Recarga automática:  No
- ♦ Ideal para clase:  Solo para proyectos terminados
- ♦ Uso principal: Servir archivos estáticos
- ♦ URL por defecto:  `http://localhost:5000`
- ♦ Instalación: `npm install serve --save-dev` o `-g`
- ♦ Script npm: `"start": "serve ."`

💡 *Tip:* Útil para probar o compartir un proyecto terminado.

Ejemplo de trabajar con módulos en un servidor (NPM) con lite-server

Nuestro proyecto incluirá los archivos que hemos visto anteriormente:

```
modulos-js/  
├─ index.html  
├─ app.js  
└─ math.js
```



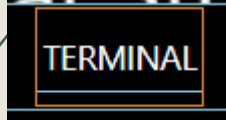
En Visual Code ...



Paso 1

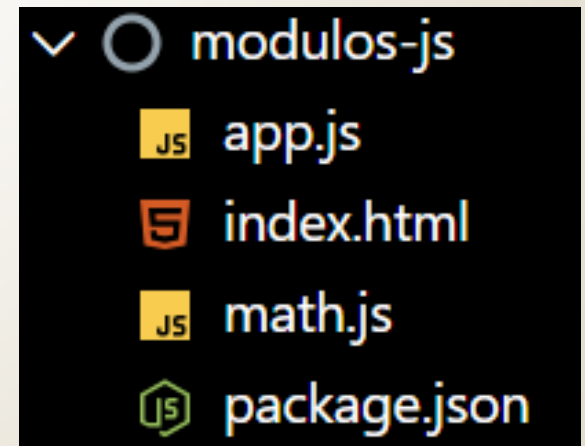
package.json es el archivo central de un proyecto Node/npm. Contiene toda la información que npm necesita para trabajar (nombre del proyecto, versión, scripts, dependencias,...):

Como no lo tenemos, hay que crearlo. Desde el terminal:



```
npm init -y
```

... esto nos crea un **package.json** mínimo con valores por defecto en nuestra carpeta:



Paso 2

Instalar **lite-server**:

```
npm install lite-server --save-dev
```

Paso 3

Editar **package.json** para añadir el script de inicio.

La sección **scripts** debe quedar así:

```
"scripts": {  
  "start": "lite-server"
```

El archivo **package.json** completo sería:



```
{
  "name": "modulos-js",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "lite-server"
  },
  "devDependencies": {
    "lite-server": "^2.6.1"
  }
}
```

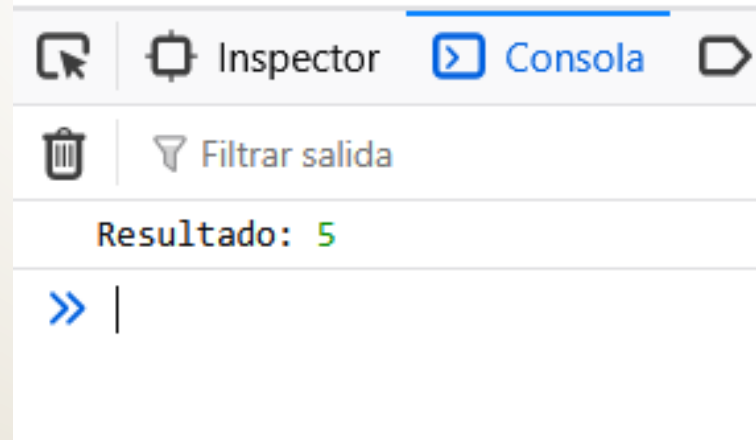
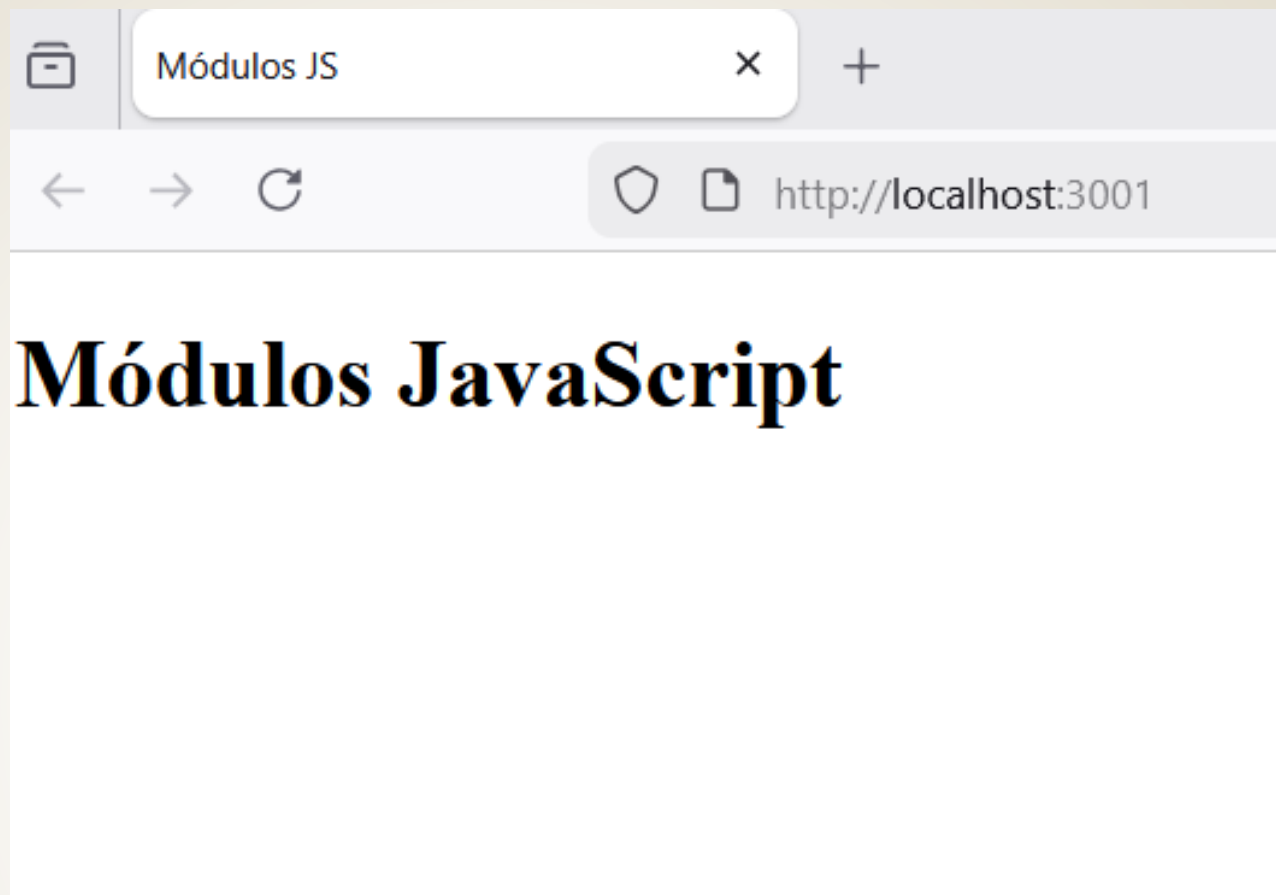
Paso 4

Arrancar el servidor

```
npm start
```

... y se abre automáticamente la página en el navegador.





Referencias. NPM cheatsheet

<https://lenguajejs.com/javascript/modulos/que-es-esm/>

https://xxjcaxx.github.io/libro_dwec/modulos.html

<https://www.luisllamas.es/npm-cheatsheet/>

<https://devhints.io/npm>



Ejemplos Módulos:

<https://github.com/mdn/js-examples.git#main>